



武汉理工大学

WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发课程报告

本地代码智能体 Harness

姓名：郭明鑫

方向：Agentic AI 原生开发

学号：2025303035

班级：研 254

专业：软件工程

指导老师：戚欣

提交日期：2026 年 6 月 22 日

一、选题背景与设计思想

1.1 选题背景

随着大语言模型（LLM）能力的持续增强，人工智能在软件工程领域的应用逐渐从传统的“问答式辅助工具”转向更具工程执行能力的“智能体系统（Agent System）”。在早期应用中，大模型主要承担代码补全、文档生成或单轮问题解答的角色，其本质仍然是一次输入对应一次输出的静态生成过程。然而在真实的软件开发场景中，任务往往具有明显的多阶段性与状态依赖性，例如在修复一个复杂 Bug 时，开发者通常需要先定位问题模块、阅读相关源码与测试用例、分析运行日志、进行代码修改并最终验证修复结果。这一完整过程并不能通过单次模型调用完成，而必须依赖持续的上下文维护与多轮决策执行能力。

因此，传统“单轮对话式”的模型交互方式在工程实践中存在明显局限，主要体现在缺乏状态记忆、无法持续执行任务以及难以与真实文件系统和开发工具进行有效协同。基于这一背景，本课程设计选取“本地代码智能体 Harness”作为研究与实现对象，旨在构建一个能够在本地代码仓库环境中持续运行的智能体系统，使大模型不再仅仅是信息生成器，而是能够在受控环境中进行分析、决策与执行的工程执行主体。

1.2 设计目标与核心思想

本项目的核心设计目标，是构建一个面向真实代码仓库的可执行智能体框架，使大模型具备在本地环境中完成复杂软件工程任务的能力。与传统基于 Prompt 的工具不同，本系统强调“运行时驱动”的设计理念，即模型本身不直接控制系统行为，而是通过外部 Runtime 统一调度其决策输出，并在严格约束下转化为可执行动作。系统的设计重点并不在于提升模型本身的能力，而在于为模型提供一个稳定、可控且可恢复的执行环境，使其能够在多轮交互中逐步完成复杂任务。

在设计思想上，本系统采用“模型解耦 + 状态外置 + 工具受控 + 过程可追溯”的整体架构。模型只负责生成决策信号，而所有状态（包括会话状态、工具执行记录、文件摘要、运行轨迹等）均由外部系统统一管理。同时，系统通过工具边界机制限制模型行为范围，使其只能在预定义的工具集合内进行操作，从而避免对本地环境造成不可控影响。此外，系统强调运行过程的可追溯性与可评估性，每一次任务执行都会生成完整的运行工件，用于后续调试、复盘与性能评估。

从整体目标来看，本项目不仅是一个代码助手工具，更是一个轻量级“代码执行型智能体框架原型”，用于验证在真实工程环境中构建可控 Agent 系统的可行路径。

1.3 设计理念总结

在工程实现层面，本系统的设计理念可以抽象为三个核心关键词：可控性、持续性与可观测性。所谓可控性，是指模型的所有行为必须经过系统显式约束，包括工具调用约束、路径访问约束以及执行权限控制；持续性是指系统必须能够支持跨轮任务执行，即在多轮交互中维持一致状态，而非每轮独立计算；可观测性则要求系统能够完整记录执行过程，使得每一步模型决策与工具行为都可以被回溯与分析。

这三者共同构成了本系统的工程设计主轴，使其区别于传统的 AI 应用系统，而更接近一个具备执行能力的“运行时智能系统”。

二、Specs 规格文档

2.1 Product Spec / Architecture Spec

本系统定义为一个面向本地代码仓库的智能体执行框架 **Harness**，其核心目标是在真实工程环境中为大语言模型提供一个可持续运行的“执行外壳”，使模型能够在代码仓库中完成从问题理解到代码修改再到结果验证的完整工程链路。与传统意义上的 AI 编程助手不同，本系统并不将模型视为最终执行主体，而是将其嵌入到一个受控运行时环境中，通过系统级调度实现模型能力的工程化落地。

在产品定义层面，该系统的输入为自然语言形式的开发任务描述，例如“修复测试失败问题”或“优化某个模块逻辑”，输出则不再是单一文本结果，而是包含代码变更、工具执行记录以及结构化运行报告的完整任务执行结果。系统运行过程中，模型需要基于当前仓库状态进行多轮推理，每一轮推理可能触发文件读取、代码修改、命令执行等工具操作，而所有这些行为均由运行时系统统一调度与约束。

从架构设计来看，本系统采用典型的分层式 **Agent Runtime** 架构，整体由模型适配层、上下文管理层、工具执行层、运行时控制层以及持久化层共同构成。模型适配层负责屏蔽不同大模型 API 的差异，使系统能够兼容多种模型后端；上下文管理层负责构建 **prompt** 输入结构，将系统规则、工作区信息、记忆与历史记录进行统一组织；工具执行层则定义了模型可操作的有限动作空间，并通过 **schema** 校验与权限控制确保执行安全；运行时控制层是系统的核心调度单元，负责驱动多轮推理循环；持久化层则用于记录 **session** 状态与 **run** 级别的执行工件，使系统具备可恢复与可回溯能力。

该架构的关键设计思想在于将“模型能力”与“执行能力”彻底解耦，使模型仅承担决策角色，而所有现实世界的副作用均由系统运行时统一管理，从而保证整个系统在复杂工程环境中的稳定性与安全性。

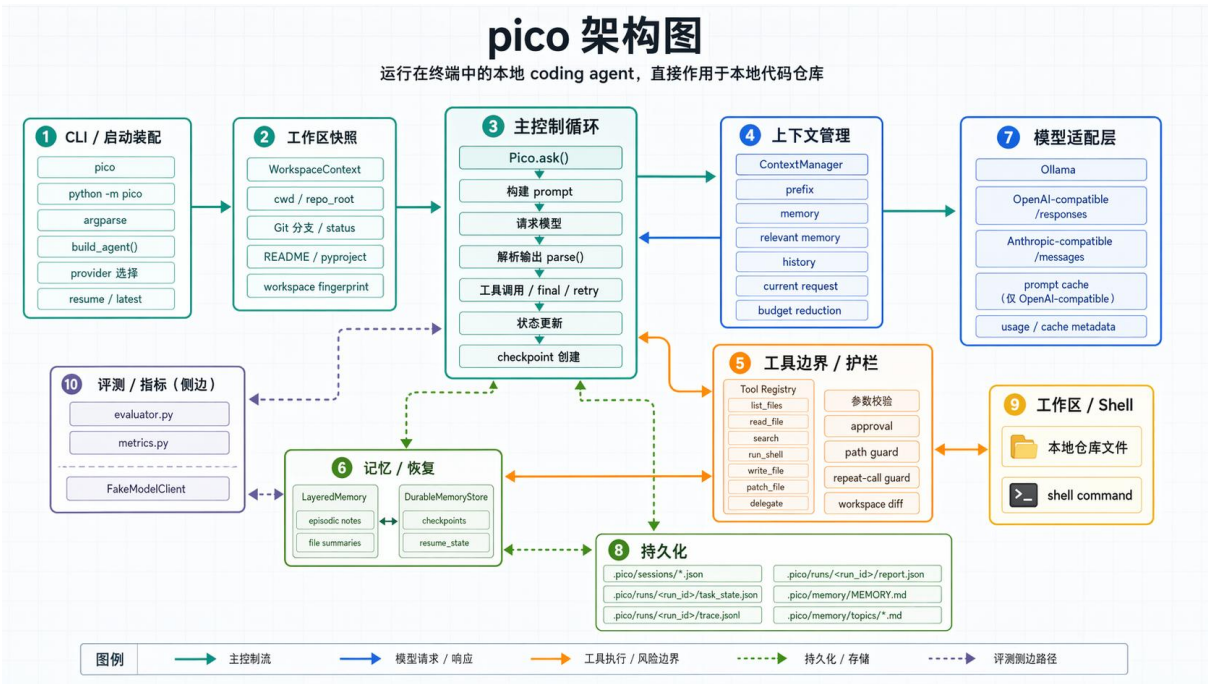


图 1：系统分层架构图

2.2 API / Execution Spec

在执行接口设计层面，本系统以 `runtime.ask()` 作为核心执行入口，该接口抽象了整个智能体运行生命周期，使每一次用户请求都能够被封装为一个独立的执行单元 `run`。系统并不直接执行单轮 `Prompt`，而是将其纳入完整运行时上下文中进行处理，从而支持多轮交互与状态延续。

当用户输入一个自然语言任务后，系统首先会创建任务上下文，并初始化 `TaskState` 对象，用于记录当前任务的执行进度、工具调用次数以及运行状态。随后系统进入主循环，在每一轮迭代中构建完整 `prompt` 输入，该 `prompt` 不仅包含用户当前请求，还会动态融合系统规则、工作区信息、历史对话记录以及结构化记忆内容，从而为模型提供完整的决策上下文。

模型返回结果后，系统不会直接执行，而是先通过解析器对输出进行结构化判断，识别其属于工具调用（`tool call`）、最终结果（`final answer`）还是无效输出（`retry`）。如果识别结果为工具调用，则进一步进入工具执行流程；如果为最终结果，则结束当前任务；如果为无效输出，则触发重试机制重新进入推理循环。

在工具调用阶段，系统会严格根据工具定义的 `schema` 对输入参数进行校验，同时结合运行时安全策略进行权限判断，确保高风险操作（如文件写入或 `shell` 执行）必须经过显式审批后才能执行。执行完成后，工具结果会被写入 `trace` 日志与状态系统，并反馈到下一轮 `prompt` 构建中，从而形成闭环式执行逻辑。

整体而言，该 API 设计并非传统意义上的函数调用接口，而是一个完整的“运行时驱动执行模型”，其核心价值在于将大模型的生成能力转化为可控的工程执行流程。

2.3 SDD (Software Design Description)

在软件设计描述层面，本系统的核心原则是“模型无直接执行权”，即模型不具备直接访问文件系统或执行系统命令的能力，而必须通过预定义工具接口进行间接操作。这一设计从根本上避免了模型在不确定输出情况下对系统环境造成不可控影响的问题。

所有工具均通过 `Tool Registry` 进行统一管理，每一个工具在注册时必须显式定义其功能描述、参数结构（`schema`）以及风险等级（`risk level`）。运行时系统在接收到模型的工具调用请求后，会首先对其进行合法性校验，包括工具是否存在、参数是否合法以及调用频率是否异常等多重检查。在此基础上，对于标记为高风险的工具，还会触发额外的审批流程，从而形成“解析—校验—审批—执行”的完整安全链路。

与此同时，系统引入 `Workspace Context` 作为运行环境抽象层，用于描述当前代码仓库的整体状态，包括当前工作目录、Git 分支信息、未提交修改状态以及关键项目文档摘要等内容。该上下文在每一轮 `prompt` 构建时都会被动态注入，从而保证模型始终基于最新的仓库状态进行决策，而不会依赖过时信息。

从设计角度来看，SDD 层的核心作用在于定义系统的“行为边界”，即明确模型可以做什么、不能做什么，以及所有行为必须如何被记录与约束。通过这一层设计，系统在保证模型灵活性的同时，也实现了工程级别的安全控制与行为规范。

三、系统架构与设计

3.1 整体系统架构

本系统在整体设计上采用分层式智能体架构，其核心目标是在本地代码仓库环境中构建一个可持续运行、状态可控且行为可追踪的代码执行型智能体系统。与传统基于单一模型调用的应用不同，本系统并不依赖模型本身完成全部任务，而是通过运行时系统（Runtime）对模型行为进行统一调度，从而将其纳入一个结构化的软件执行框架中。

从系统结构来看，整个 Harness（Pico）可以抽象为四个核心层次：模型层、控制运行层、上下文与状态层以及工具执行与持久化层。模型层负责生成决策输出，但不直接参与系统执行；控制运行层作为系统核心调度中心，负责驱动多轮推理循环；上下文与状态层用于维护系统运行过程中所有必要信息，包括会话状态、记忆信息以及代码仓库上下文；工具执行与持久化层则负责将模型的决策转化为实际的文件操作、命令执行以及运行记录存储。

这种分层结构的核心价值在于实现模型与系统执行能力的彻底解耦，使模型仅作为“决策引擎”存在，而所有现实世界副作用（如文件修改、命令执行等）均由系统统一控制，从而保证整体执行过程的安全性及可控性。同时，该架构也为系统的可扩展性提供了基础，使得未来可以在不修改模型逻辑的情况下扩展工具能力或增强运行时策略。

从运行视角来看，用户输入的自然语言任务首先进入 CLI 启动层，然后被封装为运行时任务（Run），随后进入 Pico Runtime 主循环，在循环中不断进行“上下文构建 → 模型调用 → 输出解析 → 工具执行 → 状态更新”的迭代过程，直到任务完成或达到终止条件为止。

3.2 主控制循环设计（Runtime Loop）

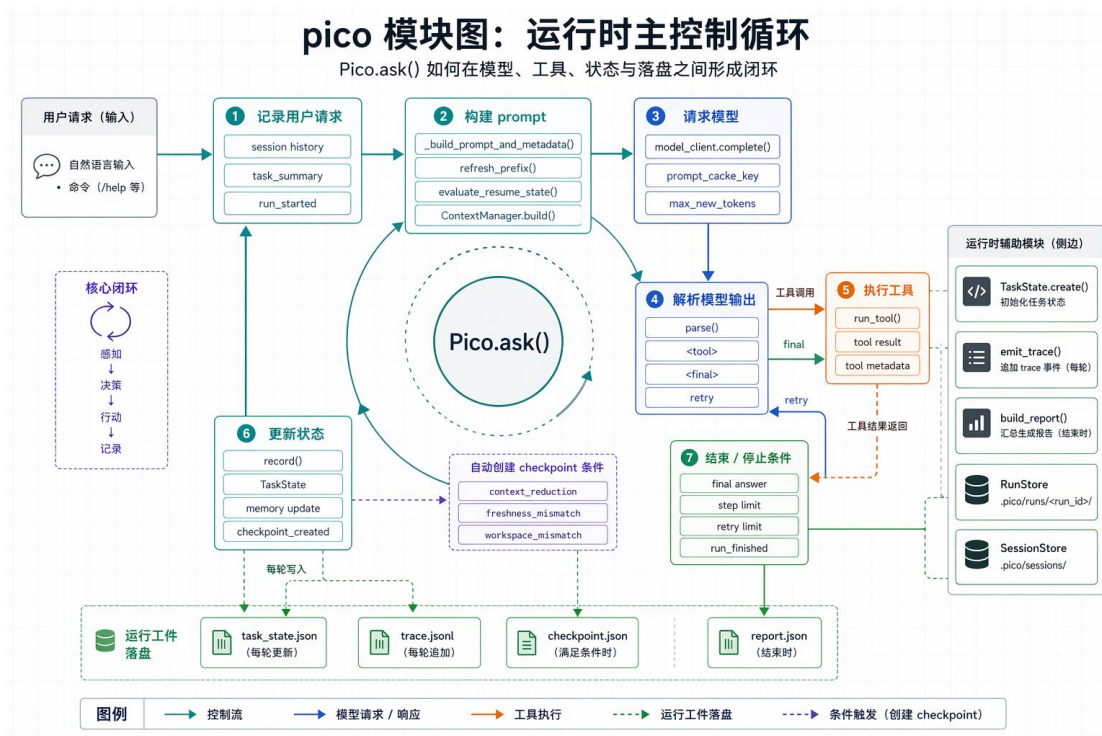
本系统最核心的执行机制是 `Pico.ask()` 所驱动的主控制循环，该循环构成了整个智能体系统的“执行心脏”。与传统程序的一次性执行不同，该循环采用多轮迭代驱动的方式，使模型能够在每一轮中逐步完善对任务的理解，并通过工具调用不断推进任务状态。

在每一轮循环开始时，系统首先会重新构建完整的 `prompt` 输入，该 `prompt` 并不是静态拼接结果，而是基于当前系统状态动态生成的结构化上下文，其中包含系统规则、工具说明、当前工作区信息、历史交互记录以及结构化记忆内容。这种设计确保模型在每一轮决策时都基于“最新状态”，而不是依赖过期信息。

模型输出后，系统不会直接执行其内容，而是先通过解析器对输出进行结构化分类，判断其属于工具调用、最终答案还是无效输出。如果输出为工具调用，则进入工具执行阶段；如果为最终答案，则结束当前任务；如果为无效输出，则触发重试机制重新进入循环。

工具执行完成后，其结果会被写入状态系统，同时生成 `trace` 记录，并反馈回下一轮 `prompt` 构建流程，从而形成一个闭环反馈结构。整个主循环本质上构成了一个“感知—决策—执行—反馈”的持续优化系统，使模型具备逐步完成复杂工程任务的能力。

该设计的关键优势在于，它将传统一次性生成模型转变为具备过程控制能力的执行系统，使复杂任务能够通过多轮分解逐步完成，而不是依赖单次推理能力。



3.3 上下文管理与状态系统设计

在复杂代码任务场景中，模型性能的关键瓶颈并不完全来自模型能力本身，而是来自上下文管理能力。因此，本系统专门设计了 Context Manager 与 Memory System，用于对输入信息进行结构化组织与动态控制。

Context Manager 的核心职责是构建每一轮模型输入的 prompt 结构，其设计原则是将上下文划分为“稳定部分”和“动态部分”。其中稳定部分包括系统行为规则、工具定义以及工作区摘要，这些内容在多轮执行过程中变化较少；动态部分则包括当前用户请求、历史交互记录以及运行过程中的记忆信息，这些内容会随着任务推进不断变化。

在 memory 层面，系统引入轻量级结构化记忆机制，用于记录跨轮有效信息，例如任务摘要、最近访问文件以及文件级摘要信息。这些信息并不是简单的历史记录，而是经过压缩与结构化后的“任务关键状态”，用于避免模型在多轮执行过程中重复读取相同信息或丢失关键上下文。

同时，系统通过 fingerprint 机制对 workspace 状态进行标识，当检测到代码仓库状态发生变化（例如分支切换、文件修改或关键文档变更）时，系统会触发上下文刷新机制，重新生成稳定前缀，从而确保模型始终基于最新环境进行决策。

这一设计的核心意义在于，它避免了传统长上下文模型中常见的信息冗余与上下文污染问题，使系统能够在有限 token 预算下保持高质量决策能力。

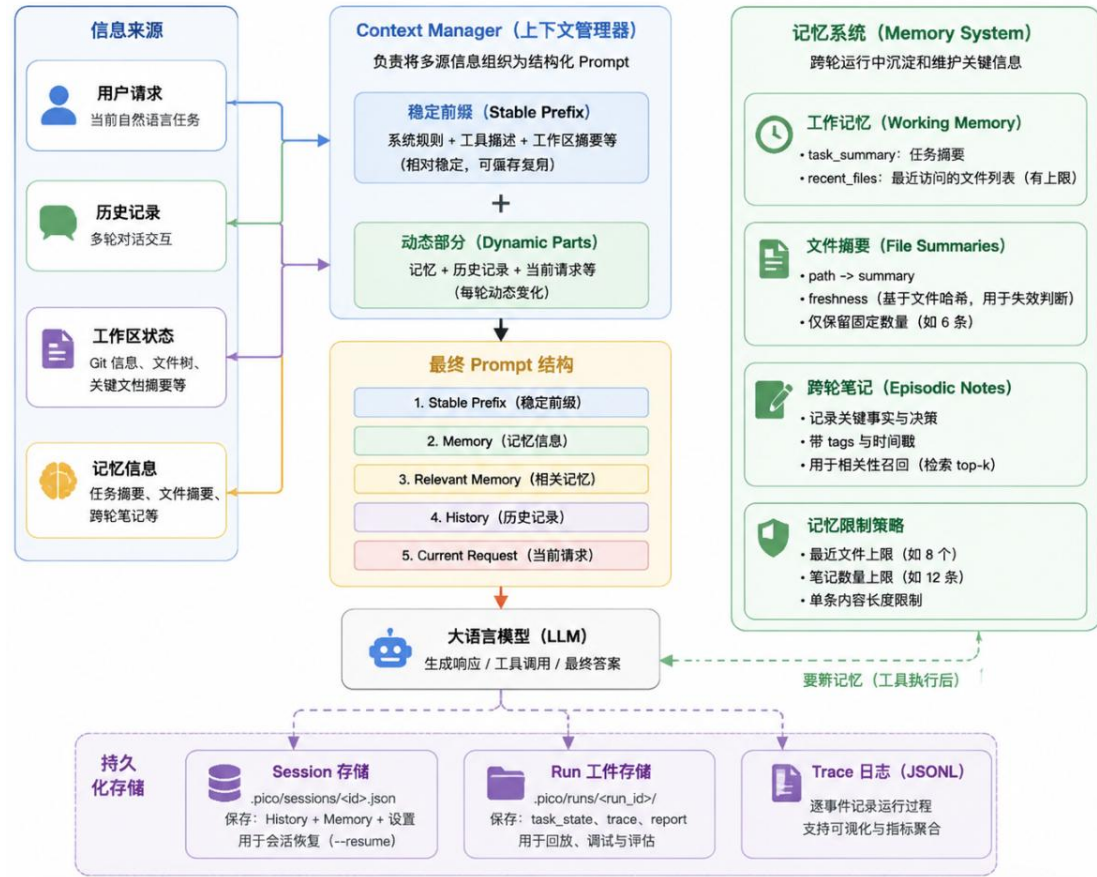


图 3：上下文与记忆结构图

3.4 工具执行与安全控制架构

工具系统是本智能体框架中最关键的执行组件，其设计直接决定了模型是否具备真实工程操作能力，同时也决定了系统的安全边界是否可控。

在本系统中，所有工具均通过 Tool Registry 进行统一注册与管理，每个工具必须显式定义其功能描述、输入参数结构（schema）以及风险等级（risk level）。模型不能直接调用系统 API，而必须通过结构化工具调用接口发出请求，由运行时系统进行统一解析与调度。

当模型发出工具调用请求后，系统会依次经过工具存在性检查、参数合法性验证以及执行安全校验三个阶段。对于高风险工具（如文件写入、shell 命令执行等），系统还会额外触发人工或策略级审批机制，以防止模型执行不可控操作。

在执行过程中，系统还会对重复调用进行检测，避免模型在无效路径上反复执行相同操作，从而提高整体运行效率。执行结果会被写入 trace 系统，同时更新 memory 与 task state，使其能够影响后续 prompt 构建。

这一整套工具执行链条的设计核心在于建立“强约束执行边界”，即模型可以提出任何合法工具调用，但所有真实系统行为必须经过运行时验证，从而确保系统既具备灵活性，又不会失去安全性。

四、关键实现与代码展示

4.1 CLI 启动与运行时装配实现

本系统的启动入口位于 CLI 层，其核心职责是将用户输入的自然语言任务转换为一个完整的运行时执行环境。在系统启动阶段，CLI 并不直接进入模型调用，而是首先构建工作区上下文（WorkspaceContext）、会话存储（SessionStore）以及模型适配客户端（ModelClient），从而形成完整的运行时依赖图。

这一设计的关键意义在于：系统并不是“启动即执行”，而是先完成运行环境装配，再进入智能体执行阶段，从而保证后续所有行为都在统一上下文中运行。

关键代码如下：

```
def build_agent(args):
    workspace = WorkspaceContext.build(args.cwd)
    store = SessionStore(
        workspace.repo_root + "/.pico/sessions"
    )
    model = _build_model_client(args)
    session_id = args.resume
    if session_id == "latest":
        session_id = store.latest()
    return Pico(
        workspace=workspace,
        store=store,
        model=model,
        session_id=session_id,
        max_steps=args.max_steps,
    )
```

从实现上可以看到, CLI 层的核心不是逻辑执行,而是“系统装配器(System Bootstrapper)”。它将运行所需的所有组件提前初始化,使后续 runtime 可以专注于任务执行本身。

```
pico — python · uv run pico — 127x35
```

```
[martinlos@192 pico % uv run pico
```

```
+=====+
|                                     |
|                                     |
|                                     |
|          /\_ _ /\_ _              |
|         (  o o  )                 |
|           ^                       |
|        /\_ _ /\_ _               |
|       /  |  |  |  |              |
|      pico                                     |
|      local coding agent                 |
|      calm shell, ready for work        |
|                                     |
+=====+

| WORKSPACE   /Users/martinlos/code/pico |
| MODEL       gpt-5.4                   |
| APPROVAL    ask                       |
| BRANCH      main                      |
| SESSION     20260409-212105-b5a2de   |
|                                     |
+=====+
```

```
pico> 看下这个代码仓库是在干什么
```

这个仓库是在做一个本地代码助手 `pico`。

- 它是面向“当前代码仓库”工作的轻量 coding agent，不是普通聊天工具
- 运行形态是命令行工具，包名/CLI 都叫 `pico`，也支持 `python -m pico`
- 主要用途是：读仓库、分析代码、排查问题、修改文件、执行命令、修测试
- 它会将会话和运行产物持久化到本地 `.pico/` 目录，方便连续工作
- 支持多种模型后端：Ollama、OpenAI 兼容 Responses API、Anthropic 兼容 Messages API

从 README 看，核心定位可以理解为：
“一个直接在终端里、围绕本地仓库持续工作的 coding agent”。

如果你愿意，我下一步可以继续帮你看两件事之一：

1. 梳理这个仓库的代码结构和核心模块
2. 说明它一次完整运行的流程（CLI → 会话 → 工具调用 → 持久化）

```
pico> █
```

图 4: 运行部分截图

4.2 主控制循环实现（Agent Runtime Core）

系统的核心逻辑集中在 `Pico.ask()` 方法中，该方法构成整个智能体系统的执行引擎。其本质是一个多轮迭代控制循环，通过不断调用模型、解析输出以及执行工具来推进任务完成。

该循环体现了典型的 `Agentic Execution Pattern`，即“感知—推理—行动—反馈”的持续闭环。

关键代码如下：

```
def ask(self, user_message):
    task_state = TaskState.create(
        run_id=self.new_run_id(),
        task_id=self.new_task_id(),
        user_request=user_message,
    )
    self.current_task_state = task_state
    self.current_run_dir = self.run_store.start_run(task_state)
    self.record({"role": "user", "content": user_message})
    attempts = 0
    tool_steps = 0
    while attempts < self.max_steps:
        attempts += 1
        task_state.record_attempt()
        prompt, metadata = self._build_prompt_and_metadata(user_message)
        raw = self.model_client.complete(prompt)
        kind, payload = self.parse(raw)
        if kind == "tool":
            tool_steps += 1
            result = self.run_tool(
                payload["name"],
                payload.get("args", {})
            )
            self.record_tool(payload, result)
            self.emit_trace("tool_executed", payload, result)
            continue
        if kind == "retry":
            self.emit_trace("retry", {"raw": raw})
            continue
        task_state.finish_success(payload)
        self.run_store.write_report(task_state)
        return payload
    task_state.finish_failed()
    return None
```

这一段代码体现了系统最核心的设计思想：模型不是执行者，而是决策器；执行由 `runtime` 控制。

同时可以看到，系统通过 `attempts` 与 `tool_steps` 双重计数机制控制执行节奏，避免无限循环或工具滥用。

4.3 Prompt 构建与上下文管理实现

在本系统中，`prompt` 并不是简单的字符串拼接，而是由 `ContextManager` 动态构建的结构化上下文，其核心目标是将系统状态转化为模型可理解的输入表示。

`prompt` 主要由四个部分组成：稳定前缀（`prefix`）、记忆（`memory`）、历史记录（`history`）以及当前请求（`current request`）。其中 `prefix` 是相对稳定部分，包含系统规则、工具说明与工作区摘要；其余部分均为动态内容。

关键代码如下：

```
def _build_prompt_and_metadata(self, user_message):
    prefix = self.context_manager.build_prefix()
    memory_text = self.memory.render()
    history_text = self.render_history()
    prompt = "\n\n".join([
        prefix.text,
        memory_text,
        history_text,
        f"User Request:\n{user_message}"
    ])
    metadata = {
        "prefix_hash": prefix.hash,
        "workspace_fp": prefix.workspace_fingerprint,
        "history_len": len(history_text),
    }
    return prompt, metadata
```

这一设计的关键点在于“分层构建”，即不同类型的信息具有不同生命周期。`prefix` 相对稳定，可复用；`memory` 与 `history` 动态变化，需要每轮更新；用户请求则始终作为最新输入存在。

通过这种结构化方式，系统避免了传统 `prompt` 拼接中的混乱与冗余问题。

4.4 工具执行系统实现（Tool Runtime）

工具系统是整個智能体框架中最关键的执行模块，其核心职责是将模型输出的结构化意图转化为真实系统行为。本系统通过 `Tool Registry` 管理所有可用工具，并在执行前进行严格校验与安全控制。

工具注册结构如下：

```
TOOLS = {
    "read_file": {
        "schema": {"path": "str"},
        "risky": False,
    },
    "write_file": {
        "schema": {"path": "str", "content": "str"},
        "risky": True,
    },
}
```

```

    },
    "run_shell": {
        "schema": {"cmd": "str"},
        "risky": True,
    }
}

```

工具执行核心逻辑如下：

```

def run_tool(self, name, args):
    tool = TOOLS.get(name)

```

```

    if tool is None:
        return {"error": "unknown tool"}

```

```

    self.validate(tool, args)

```

```

    if tool["risky"]:
        if not self.approve(name, args):
            return {"error": "denied"}
    result = tool_impl[name](args)
    self.update_memory(name, args, result)
    self.write_trace(name, args, result)
    return result

```

这一层设计的核心在于“三段式控制结构”：

首先进行工具合法性检查，其次进行安全审批控制，最后才进入真实执行阶段，从而保证系统不会因为模型输出错误而产生不可控副作用。

4.5 状态持久化与运行工件生成

系统在每一次 `ask()` 执行结束后，都会生成完整的运行工件 (Run Artifacts)，用于支持后续调试、回放与评估。这些工件包括 `task_state`、`trace` 日志以及 `report` 文件。

关键代码如下：

```

def write_report(self, task_state):
    report = {
        "task_id": task_state.task_id,
        "status": task_state.status,
        "attempts": task_state.attempts,
        "tool_steps": task_state.tool_steps,
        "final": task_state.final_answer,
    }

```

```

    path = self.run_dir / "report.json"

```

```

    path.write_text(json.dumps(report, indent=2))

```

同时，`trace` 采用 JSONL 方式逐条写入，以保证运行过程的可追踪性：

```

def emit_trace(self, event_type, data):
    record = {

```

```
        "event": event_type,
        "data": data,
        "timestamp": now()
    }
    with open(self.trace_path, "a") as f:
        f.write(json.dumps(record) + "\n")
```

这种设计使系统具备天然的“可观测性（Observability）”，可以完整复现每一步执行过程。

五、测试与评估

5.1 测试方法设计（Benchmark 驱动评测体系）

本系统的测试与评估部分采用 **benchmark** 驱动的系统化评测方法，而非传统意义上的功能性测试。其核心目标不是验证某个函数是否正确执行，而是验证整个智能体系统在真实代码仓库环境中是否具备稳定的任务执行能力。因此，测试对象从“代码函数”上升为“端到端任务执行能力”。

在该评测体系中，每一个测试任务都被抽象为一个独立的执行合同（**Execution Contract**），其中不仅包含任务描述，还明确规定了任务执行环境、允许使用的工具集合、最大执行步数以及最终结果验证方式。这种设计使得每一个任务都可以被重复执行，从而具备可复现性与可对比性。

系统在执行测试时，会将每一个 **benchmark** 任务加载到独立的 **fixture** 仓库中，并在隔离环境中启动智能体运行实例，使其在受控代码环境中完成任务。通过这种方式，可以有效避免不同任务之间的状态污染问题，同时保证评测结果的独立性与一致性。

此外，为了进一步降低模型随机性对评测结果的影响，系统在测试阶段引入 **FakeModelClient**，用于提供确定性输出，从而使测试重点聚焦于 **runtime** 系统本身，而非模型能力波动。

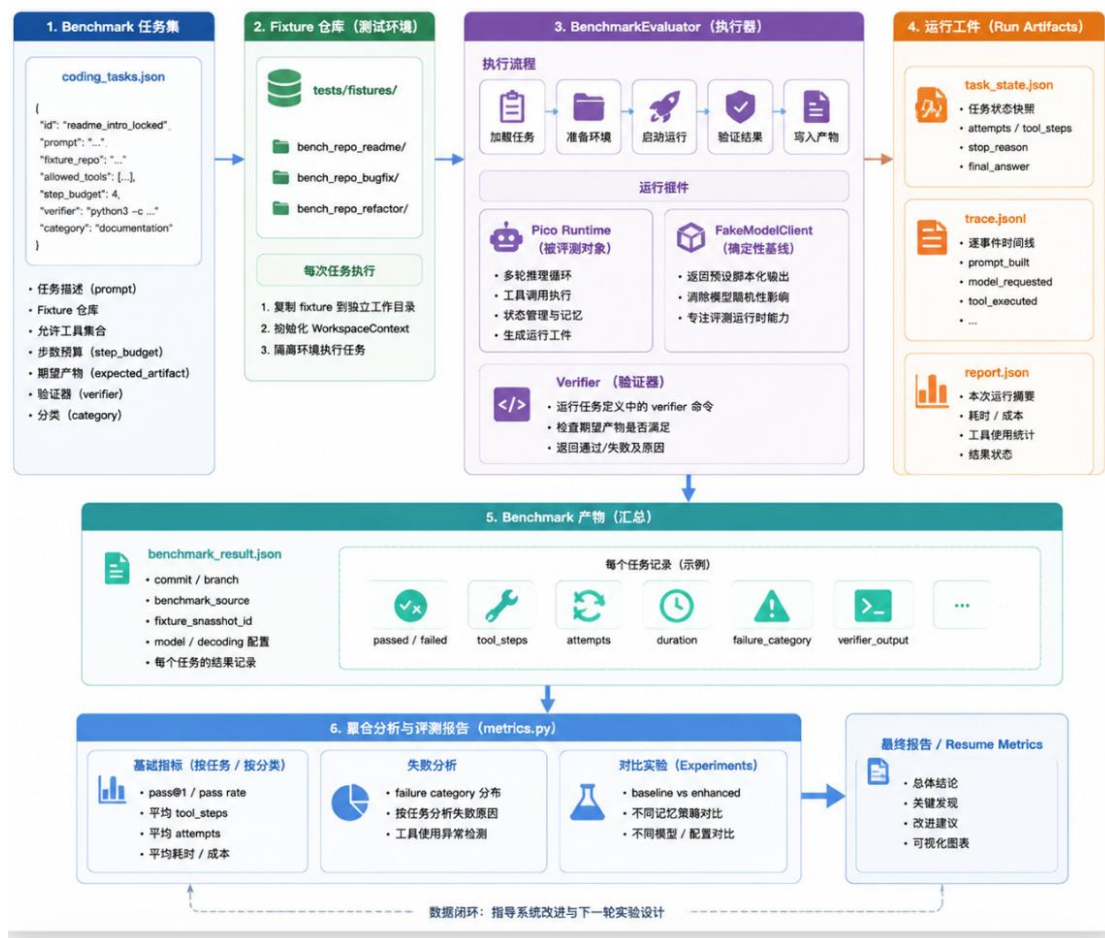


图 5：评测体系结构图

5.2 测试流程与执行机制

系统的测试执行流程由 **BenchmarkEvaluator** 统一管理，其核心流程可以分为四个阶段：任务加载阶段、环境初始化阶段、执行阶段以及验证阶段。

在任务加载阶段，系统从 **benchmark** 配置文件中读取测试任务定义，包括任务描述、工具限制、预期输出以及验证器脚本。在环境初始化阶段，系统会将对应的 **fixture** 仓库复制到独立工作目录，并在该目录中初始化 **WorkspaceContext**，使其与真实运行环境保持一致但彼此隔离。

在执行阶段，系统启动 **Pico Runtime**，并将任务输入传递给智能体执行循环。智能体在该阶段通过多轮推理不断调用工具并修改代码状态，直至达到停止条件。在该过程中，系统会完整记录 **trace**、**task_state** 以及所有工具执行结果。

在验证阶段，系统会通过预定义 **verifier** 对最终结果进行自动验证，例如检查文件内容是否正确修改、测试脚本是否通过或特定代码结构是否存在。只有当所有验证条件同时满足时，该任务才被判定为通过。

5.3 评估指标设计（Evaluation Metrics）

本系统的评估体系不仅关注最终任务是否完成，还关注执行过程的效率性与稳定性。因此，评估指标被设计为多维度组合指标，而非单一成功率指标。

首先是任务完成率（**Success Rate**），用于衡量系统在 **benchmark** 任务中的整体完成能力。该指标要求任务必须同时满足验证器通过、未超出工具调用预算以及正常终止条件。

其次是工具调用效率（**Tool Efficiency**），用于衡量智能体在完成过程中是否存在冗余操作。该指标通过统计 **tool_steps** 与 **attempts** 的比例进行衡量，用于反映系统是否具备有效的执行策略，而非盲目试错。

第三是预算利用率（**Budget Utilization**），用于衡量模型是否在允许的 **step budget** 内完成任务。该指标能够反映 **runtime** 控制能力是否有效约束模型行为。

第四是失败类型分布（**Failure Categorization**），用于分析系统失败原因，包括 **verifier failure**、**artifact missing**、**budget exceeded** 等类型。该指标对于系统优化具有重要指导意义，因为它能够明确系统瓶颈究竟来自模型能力、工具链还是上下文管理。

5.4 实验设计与对照分析

在实验设计层面，本系统采用对照实验方式验证不同 **runtime** 机制对整体性能的影响。通过引入 **FakeModelClient** 与真实模型两种执行环境，可以分别评估系统结构能力与模型能力的贡献。

在 **baseline** 实验中，仅使用基础 **runtime loop**，不引入 **memory** 优化与 **prompt cache** 机制。在该条件下，系统主要依赖原始 **prompt** 构建能力进行任务执行，容易出现重复文件读取与冗余工具调用现象。

在增强实验中，引入 **ContextManager**、**Memory System** 以及 **Prompt Prefix Cache** 机制后，系统在相同任务集合上的表现显著提升。主要变化体现在工具调用次数减少、任务完成时间缩短以及失败类型中“**budget exceeded**”占比下降。

此外，通过引入 **fixture snapshot** 机制，系统可以确保每一次实验在完全一致的代码环境中运行，从而避免由于环境差异导致的实验偏差，使评估结果更具可信度。

5.5 评估结果分析与系统意义

从整体评估结果来看，本系统的核心提升并不体现在模型能力增强，而是体现在运行时系统对模型行为的结构化约束能力上。通过引入多层上下文控制机制与工具执行约束机制，系统显著降低了无效工具调用比例，并提升了任务执行的稳定性。

同时，**trace** 与 **run artifact** 的引入使得系统具备完整的可观测能力，使每一次任务执行都可以被回放与分析。这种能力对于后续系统优化具有重要意义，因为它使得优化方向不再依赖经验判断，而是基于具体执行数据进行分析。

从工程角度来看，该评测体系不仅用于验证系统当前能力，更重要的是为后续版本迭代提供可量化依据，使系统演进具备明确的数据驱动路径。

六、系统升级与扩展

6.1 可扩展方向设计

本系统在当前实现中已经具备完整的代码仓库智能体执行能力，但从工程演进角度来看，其架构仍然处于“轻量级运行时原型阶段”。因此，在后续发展中，可以从多个方向进一步扩展系统能力，使其逐步接近工业级 **Agent Runtime** 系统。

首先在工具系统层面，可以引入插件化工具注册机制，使工具从当前的静态注册表模式扩展为动态加载模式，从而支持第三方工具扩展与业务场景定制化能力。这一升级可以显著增强系统在不同代码库环境中的适配能力，使其不再局限于固定工具集合。

其次在运行时层面，可以引入多 Agent 协同执行机制，使多个 Pico 实例能够在同一代码仓库中协作完成复杂任务，例如一个 Agent 负责分析问题，另一个 Agent 负责代码修改，第三个 Agent 负责测试验证。这种分工机制可以显著提升复杂任务处理能力，同时降低单个模型推理压力。

此外，在上下文管理层面，可以进一步引入基于语义检索的长期记忆机制，例如 embedding-based retrieval，用于替代当前轻量 keyword matching 机制，从而提升复杂语义场景下的记忆召回能力，使系统能够更好地处理跨长周期的工程任务。

6.2 性能优化与工程强化方向

在性能优化层面，当前系统仍然存在一定的优化空间，主要集中在 prompt 构建效率、工具调用成本以及上下文冗余控制三个方面。

在 prompt 构建方面，可以进一步引入更细粒度的缓存机制，例如基于 prompt segment 的 partial caching，从而避免每一轮重复构建相同稳定前缀，提高整体运行效率。

在工具调用方面，可以通过引入执行批处理机制，将多个低成本工具调用合并为一次执行，从而减少 runtime 调度开销，提高执行效率。

在上下文控制方面，可以进一步优化 memory 压缩策略，使 file summary、history 与 episodic memory 之间形成更严格的层级关系，从而降低 prompt token 消耗，使系统能够在更长任务中保持稳定运行能力。

6.3 工业级系统演进路径

从更宏观的工程视角来看，本系统未来可以向“完整软件工程 AI Agent 平台”方向演进，其演进路径大致可以分为三个阶段。

第一阶段是当前的单 Agent Runtime 阶段，即以 Pico 为核心，通过单一控制循环完成任务执行。该阶段强调运行稳定性与工具控制能力。

第二阶段可以演进为多 Agent 协同系统，引入任务分解机制，使复杂任务能够被拆解为多个子任务，由不同 Agent 并行或串行执行，从而提升系统处理复杂工程问题的能力。

第三阶段则可以进一步发展为 Agent + CI/CD 深度融合系统，使智能体能够直接参与持续集成流程，在代码提交、测试执行以及自动修复环节中形成闭环，从而真正进入工程生产系统。

七、课程总结

7.1 项目总结

本课程设计围绕“本地代码智能体 Harness (Pico)”展开，从系统架构设计到运行时实现，再到评测体系构建，完整实现了一个具备代码仓库执行能力的智能体框架原型。系统的核心目标并不是简单调用大语言模型完成任务，而是构

建一个能够在真实工程环境中持续运行的 **Agent Runtime**，使模型具备状态管理能力、工具调用能力以及多轮任务执行能力。

在整个实现过程中，系统重点解决了三个核心问题：其一是如何将大模型从单轮生成工具转变为多轮执行系统；其二是如何通过上下文管理与记忆机制解决长任务中的信息衰减问题；其三是如何通过工具边界与运行时控制机制保证系统执行的安全性及可控性。

7.2 设计收获与理解提升

通过本次课程设计，可以明显体会到构建一个工程级智能体系统与传统软件开发存在本质区别。传统系统更关注确定性逻辑实现，而智能体系统则需要同时处理不确定性输出与结构化控制之间的矛盾。因此，系统设计的重点不再是单一功能实现，而是如何构建一个能够“约束不确定性行为”的运行时框架。

此外，本项目也加深了对 **Prompt Engineering**、**Context Management** 以及 **Tool Orchestration** 的理解，使人能够从系统工程角度重新理解大模型应用，而不仅仅停留在调用 **API** 的层面。

7.3 总体评价

总体而言，本项目实现了一个具备完整运行闭环的本地代码智能体系统，在架构设计上实现了模型与运行时的解耦，在工程实现上完成了工具系统、上下文管理与状态持久化的统一设计，并通过评测体系验证了系统在真实任务中的可执行能力。

该系统不仅具有课程设计意义，同时也具备一定的研究参考价值，为后续构建更复杂的 **AI Agent** 系统提供了基础架构原型与设计思路。